MycoFlow — Bio-Inspired Reflexive QoS System for OpenWrt Routers

Project Proposal Report

Student: Barış Can Ataklı
Advisor: Dr. Öğr. Üyesi Cihat Çetinkaya
Department: Software Engineering, Muğla Sıtkı Koçman University

November 2025

Contents

1 Introduction and Motivation **4**

2 Problem Statement **4**

3 Background and Related Work **5**

4 Bio-Inspired Design Rationale **6**
4.1 Mycelial Principles & Networking Analogues 6
4.2 Reflexive Control: Formal View 7
4.3 Persona Inference Heuristics 7

5 Scope and Objectives **8**
5.1 In-Scope 8
5.2 Objectives 8

6 System Architecture **9**
6.1 Reflexive Cycle Diagram (Overview) 9

7 End-to-End Lifecycle **10**
7.1 Cold Start & Bootstrap 10
7.2 Sense → Infer → Act → Stabilize 10
7.3 Audit, Telemetry, and Rollback 10
7.4 Lifecycle Pipeline Diagram 10

8 Implementation Plan and Timeline **11**

9 Algorithms and Complexity **13**
9.1 Congestion Detection 13
9.2 Actuation Scheduling 13
9.3 Resource Budget 13

10 Evaluation Methodology **14**

11 Hardware Target **14**

12 Security and Ethics **14**

13 Reproducibility and Deployment **15**
13.1 Minimal UCI 15
13.2 ubus Surface 15

13.3 Step-by-step 15

14 Future Work **16**

15 Conclusion **16**

Abstract

MycoFlow proposes a bio-inspired, fully embedded Quality of Service (QoS) framework for consumer-grade OpenWrt routers. Taking cues from mycelial networks—which dynamically allocate nutrients and communication pathways based on environmental feedback—MycoFlow implements a reflexive feedback control loop that continuously tunes QoS policies to minimize latency and jitter while preserving fairness and throughput.

The system integrates on-device telemetry (eBPF/netlink) with direct CAKE control; decisions emerge from local sensing and hysteresis-guided reactions rather than machine learning. The proposed system aims to demonstrate significant latency and jitter reduction compared to static CAKE configurations while maintaining fairness and resource efficiency on constrained hardware.

Keywords: MycoFlow, Bio-Inspired Networking, Adaptive QoS, OpenWrt, CAKE, eBPF, Reflexive Control, Persona-Aware Heuristics, Bufferbloat

1. Introduction and Motivation

Modern home networks host diverse devices—gaming consoles, smart TVs, PCs, and IoT sensors—all competing for limited bandwidth. Traditional queue management with fixed CAKE parameters fails under changing conditions. Static QoS policies are unable to account for encrypted traffic, dynamic loads, or heterogeneous device personas.

Motivation

MycoFlow brings biological adaptivity to router-level QoS. Mycelial networks excel at local sensing, distributed decision-making, and resource redistribution under stress [6, 7, 8]. MycoFlow translates these principles into network control: each router behaves as a micro-ecosystem that senses, reacts, and stabilizes autonomously.

2. Problem Statement

- **Static Configuration:** Fixed queue weights and DSCP policies cannot adapt to congestion.
- **Limited Awareness:** Port or DSCP-based classification fails under QUIC/HTTPS.
- **External Dependencies:** Reliance on remote telemetry reduces efficiency.
- **Poor Observability:** Users lack latency and queue visibility.

Goal: Design a self-contained, reflexive, persona-aware QoS system that senses real-time conditions and adjusts CAKE parameters dynamically.

3. Background and Related Work

DiffServ, CoDel, fq_codel, and CAKE algorithms mitigate bufferbloat through active queue management and fairness. Recent works extend this baseline:

- **DeepFlow (ACM SIGCOMM 2023):** Lightweight CNN for encrypted traffic classification.
- **Popescu et al. (IEEE Access 2024):** eBPF telemetry maps for adaptive edge metrics.
- **CAKE 2024:** diffserv auto-learning and per-tin delay awareness.
- **LuCI Next (2024):** WebSocket-based real-time dashboards for OpenWrt.

MycoFlow departs from ML-heavy designs, focusing on bio-inspired reflexive feedback that adapts locally without cloud intelligence.

4. Bio-Inspired Design Rationale

4.1. Mycelial Principles & Networking Analogues

Mycelial Principle	Networking Analogue in MycoFlow
Local sensing of nutrients/stress	eBPF/ICMP/UDP probes measure RTT, jitter, loss, queue depth proxies
Adaptive resource translocation	CAKE tin rebalancing, bandwidth caps, DSCP marking updates
Hysteresis & structural stability	Up/Down thresholds, stability timers, gradual parameter ramps
Distributed decision-making	Stateless rule evaluation per cycle + state memory for dampening
Network remodeling	Periodic refinement of tin shares and persona priorities

Table 1: Mycelial principles mapped to MycoFlow mechanisms.

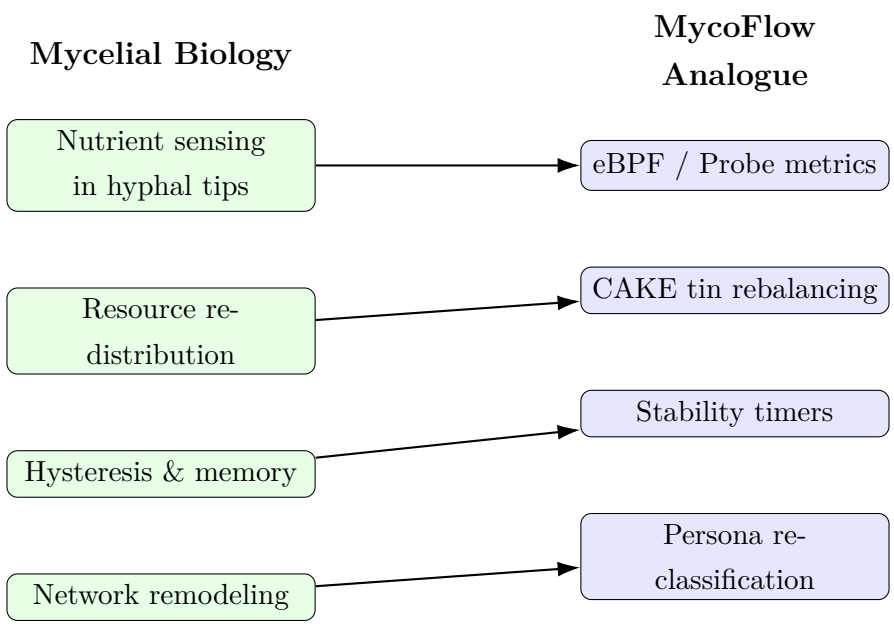

Figure 1: Mapping biological mycelial principles to MycoFlow networking concepts.

4.2. Reflexive Control: Formal View

Let x_t be the metric vector at time t (RTT, jitter, loss, utilization per persona). The target setpoint is s, and the filtered error is

$$\hat{e}_t = \alpha e_t + (1-\alpha)\hat{e}_{t-1}, \qquad e_t = x_t - s.$$

Control action:

$$\Delta u_t = \begin{cases} +k_u g(\hat{e}_t), & \hat{e}_t > \theta_{\text{up}}, \\ -k_d g(\hat{e}_t), & \hat{e}_t < -\theta_{\text{down}}, \\ 0, & \text{otherwise.} \end{cases}$$

where $g(\cdot)$ is bounded (e.g. tanh) and (k_u, k_d) are rate-limited step gains.

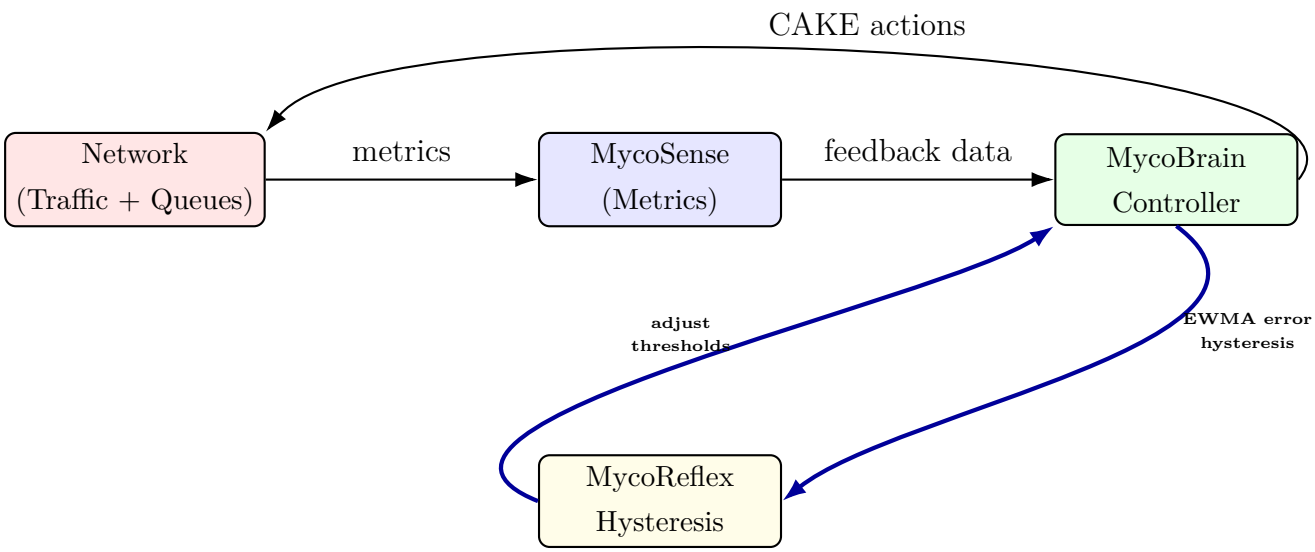

Figure 2: Reflexive feedback loop with hysteresis-based control in MycoFlow.

4.3. Persona Inference Heuristics

Signals: DNS/SNI category, packet-size histogram, inter-arrival variance, flow symmetry, time-of-day prior. Confidence accumulator:

$$c_p \leftarrow \beta c_p + (1-\beta)\phi_p(\text{signals}), \quad \beta \in [0,1)$$

Persona $\arg\max_p c_p$ with sticky hysteresis prevents rapid class flips.

5. Scope and Objectives

5.1. In-Scope

- Embedded packages (`mycoflowd`, `luci-app-mycoflow`, `kmod-myco-ebpf`)
- Reflexive feedback loop with hysteresis and rollback
- Persona-aware heuristics from DNS + flow features
- eBPF-backed telemetry with tc/netlink metrics
- LuCI (Vue-based) dashboard via ubus WebSocket

5.2. Objectives

- Reduce median RTT/jitter vs. static CAKE
- Adaptation $\leq$ 3–5 s
- CPU $\leq$ 20%, RAM $\leq$ 64 MB
- $\geq$ 90% persona accuracy

6. System Architecture

6.1. Reflexive Cycle Diagram (Overview)

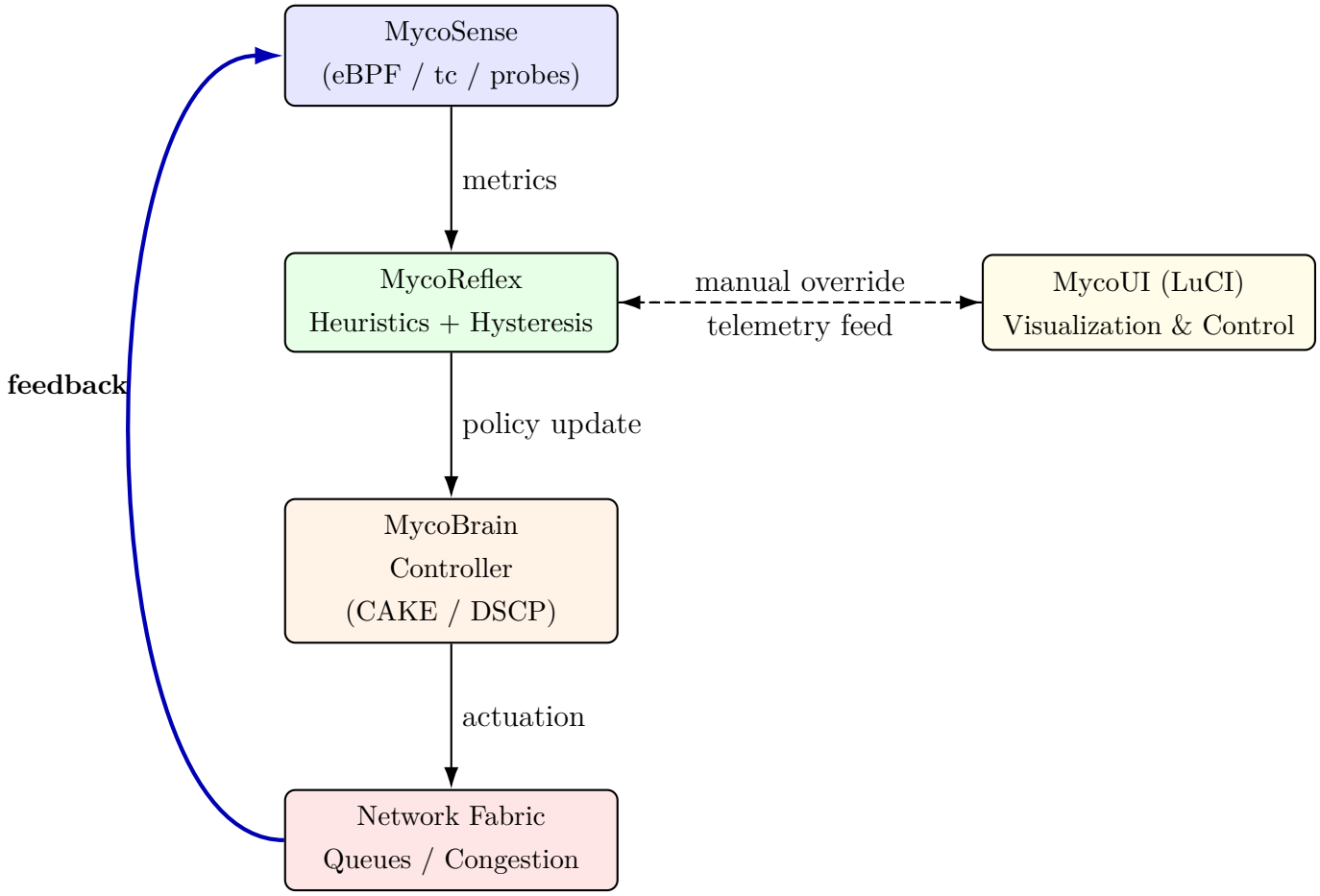

Figure 3: MycoFlow reflexive control cycle from sensing to policy actuation.

7. End-to-End Lifecycle

7.1. Cold Start & Bootstrap

1. Install packages, enable `mycoflowd`; set WAN iface and sampling rate.
2. Baseline calibration: brief probe window to estimate idle RTT and variance per tin.
3. Initialize persona priors (optional time-of-day weights).

7.2. Sense → Infer → Act → Stabilize

1. **Sense:** eBPF/Netlink collect per-flow stats; probes sample RTT/jitter.
2. **Infer:** Update persona confidences; detect congestion via thresholds on filtered metrics.
3. **Act:** Apply bounded CAKE/DSCP adjustments with ramping and cooldown timers.
4. **Stabilize:** If stable for N cycles, decay towards baseline template.

7.3. Audit, Telemetry, and Rollback

- All actions are logged with cause/metric snapshot; a sliding window of states supports rollback.
- Watchdog detects CPU spikes or metric outliers; freezes loop and reverts to last stable config.

7.4. Lifecycle Pipeline Diagram

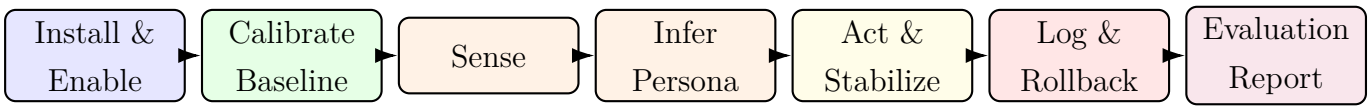

Figure 4: End-to-end lifecycle of MycoFlow deployment and operation.

8. Implementation Plan and Timeline

Table 2: Six-month implementation roadmap of the MycoFlow project (Nov 2025–May 2026).

Month / Phase	Duration	Key Deliverables and Tasks
Month 1 (Nov 2025)	Weeks 1–4	• OpenWrt installation on Xiaomi AX3000T; SSH/SCP access validation. • Buildroot toolchain and GitHub repo setup. • Literature review on CAKE, SQM, and bio-inspired networking. • Baseline latency/jitter tests via `flent` and `iperf3`. **Deliverables:** Working router, baseline dataset, CAKE analysis notes.
Month 2 (Dec 2025)	Weeks 5–8	• Develop `mycoflowd` daemon (UCI + ubus integration). • Implement CAKE parameter control via `tc`. • Add initial metrics collector (RTT, backlog, drops). **Deliverables:** mycoflowd binary, init script, CAKE control library.
Month 3 (Jan 2026)	Weeks 9–12	• Implement feedback control loop (RTT-based adjustment). • Introduce hysteresis with dual thresholds and k-of-m voting. • Add rollback and safe-mode logic. **Deliverables:** Reflexive controller, hysteresis module, rollback system.

Continued on next page

Month / Phase	Duration	Key Deliverables and Tasks (continued)
Month 4 (Feb 2026)	Weeks 13–16	• Integrate eBPF flow tracker (libbpf + TC hook). • Add Netlink statistics and real-time queue metrics. • Enhance probes with jitter/loss tracking. **Deliverables:** eBPF program, Netlink wrapper, advanced telemetry dataset.
Month 5 (Mar 2026)	Weeks 17–20	• Develop `luci-app-mycoflow` dashboard (WebSocket metrics). • Implement optional persona inference (DNS + flow features). • Conduct end-to-end stress and memory tests. **Deliverables:** LuCI app, persona engine (optional), full integration test suite.
Month 6 (Apr 2026)	Weeks 21–24	• Perform benchmark tests (gaming, conferencing, mixed load). • Tune controller parameters (α, cooldown, thresholds). • Statistical analysis and visualization of results. **Deliverables:** Evaluation dataset, tuning guide, final performance report.

Continued on next page

Month / Phase	Duration	Key Deliverables and Tasks (continued)
Month 7 (May 2026)	Weeks 25–26	• Write final thesis and prepare presentation slides. • Package release and public repository documentation. **Deliverables:** Final PDF report, LuCI screenshots, release package.

9. Algorithms and Complexity

9.1. Congestion Detection

Let r_t be RTT EWMA and j_t jitter EWMA. Congestion if:

$$r_t > r_{\text{idle}} + \delta_r \quad \vee \quad j_t > j_{\text{idle}} + \delta_j$$

with tunable margins (δ_r, δ_j); debounce via k-of-m exceedance.

9.2. Actuation Scheduling

Priority queue of pending adjustments with max rate ρ (ops/s) and min gap τ_{cooldown}. Time complexity per cycle $O(F + \log A)$, where F = active flows read from eBPF maps, A = actions.

9.3. Resource Budget

- CPU: target $< 20\%$ avg; eBPF sampling at 1–2 Hz; user space loop at 1 Hz.
- RAM: < 64 MB including flow tables; cap maps by LRU evict.
- Flash: package < 8 MB; logs rotated (size/time).

10. Evaluation Methodology

Evaluation uses `flent` and `iperf3` under scripted mixed-load profiles. Each test lasts ≥ 10 minutes with ≥ 5 repetitions for statistical confidence (t-test/ANOVA).

Scenarios: Gaming, video conferencing, mixed load, adaptive stress.

Metrics: RTT, jitter, Jain fairness, CPU/RAM.

Baselines: FIFO (no QoS), SQM/CAKE static, qosify, commercial router.

Expected: 40–70% latency reduction; jitter < 15 ms; fairness ≥ 0.9.

11. Hardware Target

Xiaomi AX3000T (MT7981B, 256 MB RAM).

- CPU usage $< 20\%$ avg (40% peak)
- RAM < 64 MB total
- Latency overhead < 2 ms
- Package size < 8 MB

12. Security and Ethics

- No payload inspection, only metadata.
- No external telemetry by default.
- Signed packages, rate-limited ubus calls; least-privilege ubus methods.

13. Reproducibility and Deployment

13.1. Minimal UCI

```
config mycoflow 'core'
  option enabled '1'
  option egress_iface 'pppoe-wan'
  option sample_hz '2'
  option max_cpu '40'
```

13.2. ubus Surface

```
myco.status    -> {policy, tins, latency}
myco.persona   -> list|add|delete
myco.policy    -> get|set|boost|throttle
```

13.3. Step-by-step

1. Install and enable packages; set WAN iface; reboot.
2. Verify eBPF maps and tc stats; run sanity probes.
3. Start traffic profiles; watch LuCI dashboard; collect logs.
4. Export CSV/JSON for analysis; compare against baselines.

14. Future Work

- Cooperative adaptation: routers exchange “nutrient maps” (aggregated telemetry).
- Optional TinyML persona prediction under strict budgets.
- SDN integration via controller APIs for policy orchestration.
- Context-aware predictive scheduling (time/location/user mode).

15. Conclusion

Bufferbloat arises from delayed feedback. MycoFlow shortens this delay through reflexive adaptation—sensing queue buildup early and adjusting CAKE weights preemptively. Each feedback cycle operates at 1 Hz, achieving sub-second reaction time. Bio-inspired by mycelial networks, MycoFlow redistributes bandwidth like nutrients—sustaining the digital ecosystem without external dependencies.

References

[1] H. Zhao et al., "DeepFlow: Lightweight Deep Learning for Encrypted Traffic Classification," Proc. ACM SIGCOMM, 2023.

[2] M. Popescu et al., "eBPF for Adaptive Network Telemetry on Edge Routers," IEEE Access, 2024.

[3] J. Nichols et al., "CAKE Enhancements for Diffserv Auto-Learning," IETF Draft, 2024.

[4] C. Tang and L. Yu, "SDN-Enabled QoS Optimization for Home Networks," IEEE ICC, 2024.

[5] OpenWrt GSOC, "LuCI Next: A Modern Web Interface for OpenWrt," 2024.

[6] M.D. Fricker, L.L.M. Heaton, N.S. Jones, L. Boddy, "The Mycelium as a Network," Microbiology Spectrum, 2017.

[7] A. Adamatzky, P. Ayres, A.E. Beasley, N. Roberts, M. Tegelaar, M.-A. Tsompanas, H.A.B. Wösten, "Logics in Fungal Mycelium Networks," Logica Universalis, 2022.

[8] Y. Fukasawa, M. Savoury, L. Boddy, "Foraging strategies of fungal mycelial networks: responses to resource heterogeneity," Frontiers in Cell and Developmental Biology, 2023.